

Tarea 1: MongoDB

Daniel Quispe Miguel
202273529-5

9 de septiembre de 2026

1. Preparación de los datos

En esta sección se declara el uso de Claude.ai para la generación de 40 datos con sus respectivos atributos. Los datos se pueden encontrar en este [repositorio](#). Cabe destacar que en este informe solo irán algunas de las capturas, la totalidad de las demás se encuentran en el repositorio.

2. Parte 1: Consultas

Con `.limit(10)` se muestra como máximo 10 documentos, además se ordenarán ascendentemente según la edad de los actores o actrices (a excepción de la ultima consulta). Por último, siempre se mostrará el campo `Nombre`, y solo se mostrarán 4 campos, siendo este último fijo para todas las consultas.

2.1. Proyección, filtrado y ordenamiento sobre campos simples.

Se utilizo la siguiente consulta:

```
db["Voice-actors"].find({
  "Edad": {$gt: 50}},
{"_id": 0, "Nombre": 1, "Nacionalidad": 1, "FechaNacimiento": 1, "Edad": 1}
).sort({"Edad": 1}).limit(10)
```

obteniendo solo 10 documentos como se puede observar en la imagen [1](#).

2.2. Búsqueda dentro de un arreglo simple.

Se utilizo la siguiente consulta:

```
db["Voice-actors"].find({
  $and: [{ "Nacionalidad": "Mexico"}, { "Idiomas": "Ingles"}]},
{"_id": 0, "Nombre": 1, "Nacionalidad": 1, "Idiomas": 1, "Edad": 1}
).sort({"Edad": 1}).limit(10)
```

Se obtuvo el siguiente resultado como se muestra en la imagen [2](#)

2.3. Búsqueda sobre arreglos de documentos embebidos.

Se utilizo la siguiente consulta:

```
db["Voice-actors"].find({
  "Edad": {$lt: 30},
  "Personajes": {
    $elemMatch: {"Rol": "principal", "Generos": "Accion"}
  }},
{"_id": 0, "Nombre": 1, "Edad": 1, "Nacionalidad": 1, "Personajes": 1
}).sort({"Edad": 1}).limit(10)
```

Como los datos de los documentos son grandes solo se dejaron algunos, las demás imágenes se pueden encontrar en el github proporcionado antes. En la imagen [3](#) se ve el resultado de un documento.

```
> db["Voice-actors"].find({
  "Edad": {$gt: 50}},
  {"_id": 0, "Nombre": 1, "Nacionalidad": 1, "FechaNacimiento": 1, "Edad": 1}
).sort({"Edad": 1}).limit(10)
< {
  Nombre: 'Hiroshi Kamiya',
  Nacionalidad: 'Japon',
  FechaNacimiento: '1975-01-28',
  Edad: 51
}
{
  Nombre: 'Takahiro Sakurai',
  Nacionalidad: 'Japon',
  FechaNacimiento: '1974-04-13',
  Edad: 52
}
{
  Nombre: 'Wanda Sequera',
  Nacionalidad: 'Venezuela',
  FechaNacimiento: '1972-02-14',
  Edad: 54
}
{
  Nombre: 'Junichi Suwabe',
  Nacionalidad: 'Japon',
  FechaNacimiento: '1972-03-29',
  Edad: 54
}
{
  Nombre: 'Ricardo Tejedo',
  Nacionalidad: 'Mexico',
  FechaNacimiento: '1971-11-11',
  Edad: 54
}
```

Figura 1: Primera consulta.

```

> db["Voice-actors"].find({
  $and: [{"Nacionalidad": "Mexico"}, {"Idiomas": "Espanol"}]},
  {"_id": 0, "Nombre": 1, "Nacionalidad": 1, "Idiomas": 1, "Edad": 1}
).sort({"Edad": 1}).limit(10)
< {
  Nombre: 'Renata Salgado',
  Nacionalidad: 'Mexico',
  Idiomas: [
    'Espanol'
  ],
  Edad: 21
}
{
  Nombre: 'Camila Ferreyra',
  Nacionalidad: 'Mexico',
  Idiomas: [
    'Espanol'
  ],
  Edad: 25
}
{
  Nombre: 'Fernanda Robles',
  Nacionalidad: 'Mexico',
  Idiomas: [
    'Espanol'
  ],
  Edad: 49
}
{
  Nombre: 'Ricardo Tejedo',
  Nacionalidad: 'Mexico',
  Idiomas: [
    'Espanol'
  ],
  Edad: 54
}

```

Figura 2: Segunda consulta.

```

> db["Voice-actors"].find({
  "Edad": {$lt: 30},
  "Personajes": {
    $elemMatch: {"Rol": "principal", "Generos": "Accion"}
  }},
  {"_id": 0, "Nombre": 1, "Edad": 1, "Nacionalidad": 1, "Personajes": 1
}).sort({"Edad": 1}).limit(10)
< {
  Nombre: 'Yuki Nakamura',
  Nacionalidad: 'Japon',
  Personajes: [
    {
      NombrePersonaje: 'Anya Forger',
      Produccion: 'Spy x Family',
      TipoProduccion: 'profesional',
      Rol: 'principal',
      Apariciones: 37,
      Generos: [
        'Comedia',
        'Accion',
        'Drama'
      ]
    }
  ],
  Edad: 22
}

```

Figura 3: Tercera consulta.

2.4. Pipeline de agregación.

Se utilizo la siguiente consulta:

```
db["Voice-actors"].aggregate([
  { $sort: { Patrimonio: 1 } },
  { $skip: 4 },
  { $limit: 10 },
  { $project:
    { _id: 0, Nombre: 1, Patrimonio: 1, Nacionalidad: 1, Edad: 1 } }
])
```

Primero se pasa por el sort para poder saltar al quinto documento y reportar los siguientes 10 documentos. Esta es la unica consulta donde el sort es en **Patrimonio**. El resultado se puede observar en la imagen 4.

3. Parte 2: Procesamiento de consultas

Se han ejecutado las siguientes consultas y guardado su información en la carpeta **explain/** en el github, más directamente [aqui](#). Con lo que luego se hacen las comparaciones mostradas en las siguientes tablas.

3.1. Ejecución sin índices

```
db["Voice-actors"].find({
  "Edad": {$gt: 50}},
{"_id": 0, "Nombre": 1, "Nacionalidad": 1, "FechaNacimiento": 1, "Edad": 1}
).sort({"Edad": 1}).limit(10).explain("executionStats")

db["Voice-actors"].find({
  $and: [{"Nacionalidad": "Mexico"}, {"Idiomas": "Ingles"}]},
{"_id": 0, "Nombre": 1, "Nacionalidad": 1, "Idiomas": 1, "Edad": 1}
).sort({"Edad": 1}).limit(10).explain("executionStats")

db["Voice-actors"].find({
  "Edad": {$lt: 30},
  "Personajes": {
    $elemMatch: {"Rol": "principal", "Generos": "Accion"}
  }},
{"_id": 0, "Nombre": 1, "Edad": 1, "Nacionalidad": 1, "Personajes": 1
}).sort({"Edad": 1}).limit(10).explain("executionStats")

db["Voice-actors"].explain("executionStats").aggregate([
  { $sort: { Patrimonio: 1 } },
  { $skip: 4 },
  { $limit: 10 },
  { $project:
    { _id: 0, Nombre: 1, Patrimonio: 1, Nacionalidad: 1, Edad: 1 } }
])
```

3.2. Índice sobre Nacionalidad

Se obtuvo los resultados al hacer la consulta 2 con y sin índices. Primero se puede apreciar que el tiempo que tardo la consulta sin indice es más rápida, aunque esto no implica que hacerlo con indice sea malo. Al tener pocos datos y al hacer un paso extra, es normal que se tarde un poco más con indice, además en la sección de **totalDocsExamined** se puede observar como paso de examinar 40 documentos a solo 10, reduciendo el trabajo de búsqueda. Otro punto importante es que las llaves examinadas sin indice son 0, pero con indice son 10, es decir, por cada entrada de mexico busca, lo que logra reducir los documentos consultados. En la Tabla 1 se puede ver las comparaciones.

```

> db["Voice-actors"].aggregate([
  { $sort: { Patrimonio: 1 } },
  { $skip: 4 },
  { $limit: 10 },
  { $project:
    { _id: 0, Nombre: 1, Patrimonio: 1, Nacionalidad: 1, Edad: 1 } }
])
< {
  Nombre: 'Daniela Ospina',
  Nacionalidad: 'Colombia',
  Patrimonio: 55000,
  Edad: 26
}
{
  Nombre: 'Yuki Nakamura',
  Nacionalidad: 'Japon',
  Patrimonio: 60000,
  Edad: 22
}
{
  Nombre: 'Camilo Henao',
  Nacionalidad: 'Colombia',
  Patrimonio: 80000,
  Edad: 30
}
{
  Nombre: 'Isabella Marchetti',
  Nacionalidad: 'Argentina',
  Patrimonio: 88000,
  Edad: 24
}
{
  Nombre: 'Camila Ferreyra',
  Nacionalidad: 'Mexico',
  Patrimonio: 95000,
  Edad: 25
}
}

```

Figura 4: Cuarta consulta.

	Sin Indice	Con Indice
Plan busqueda	COLLSCAN	IXSCAN
totalDocsExamined	40	10
nReturned	10	10
totalKeysExamined	0	10
executionTimeMillis	0	1

Cuadro 1: Tabla comparativa de trabajo sin y con indice Consulta 2.

	Sin Indice	Con Indice
Plan busqueda	COLLSCAN	IXSCAN
totalDocsExamined	40	10
nReturned	10	10
totalKeysExamined	0	10
executionTimeMillis	0	2

Cuadro 2: Tabla comparativa de trabajo sin y con indice Consulta 1.

	Sin Indice	Con Indice
Plan busqueda	COLLSCAN	IXSCAN
totalDocsExamined	40	17
nReturned	7	7
totalKeysExamined	0	17
executionTimeMillis	1	0

Cuadro 3: Tabla comparativa de trabajo sin y con indice Consulta 3.

3.3. Índice sobre Edad

Las consultas 1 y 3 son las que tienen un mayor impacto al darle un índice a las edades ya que son su criterio de búsqueda. En las Tablas 2 y 3 se puede apreciar que la cantidad de documentos examinados disminuye cuando se le integra un índice al atributo edad. Además, en la consulta 3 que es la más compleja, se puede observar que el tiempo de consulta disminuye. En la siguiente sección se comentara más sobre los planes de búsqueda y la eficiencia de tener índices.

3.4. Comparación de planes de ejecución

Como se pudo apreciar en las Tablas 1, 2 y 3, hubo un cambio de planificación a la hora de hacer las búsquedas de las consultas. Al agregar un índice tanto a **Nacionalidad** como a **Edad**, el formato de búsqueda COLLSCAN cambio a IXSCAN, que lo que hace es buscar por llaves, una vez encontrada las llaves candidatas hace un fetch para buscar los documentos completos y reportar las coincidencias. Como se observa en las tablas, la cantidad de documentos leídos son mucho menores con índice, se acerca mucho más a la cantidad de elementos que son retornados, lo que implica que no pasa por todos los documentos. La cantidad de llaves examinadas también es importante tener en cuenta, ya que al no revisar llaves, Mongo lee todo los documentos, y al tener llaves que examinar, puede reducir el espacio de búsqueda. Y por último, si bien el tiempo no disminuye como se esperaría, esto se debe que son pocos datos y pasa por una transición antes del retorno, pero con consultas más complejas si demostro ser más rápido.

3.5. Escalabilidad

Ya se pudo observar con los resultados anteriores que agregar índices mejora la búsqueda, si hay 10 mil millones de datos y no integráramos índices, entonces las consultas tendrían que revisar todos los documentos, es decir, los 10 mil millones de datos, esto haría que las consultas demoraran demasiado. En cambio, al agregar llaves, permitimos obtener candidatos y reduciendo el espacio de búsqueda considerablemente. Si tiene que procesar demasiados documentos, un servidor podría consumir todos sus recursos en obtener la respuesta, por lo que a cambio de almacenar índices, obtenemos consultas más rápidas y optimas, lo que lo hace mucho más optimo cuando trabajamos con grandes volúmenes de datos.

4. Parte 3: Flexibilidad de esquema, resiliencia y actualización

4.1. Flexibilidad de esquema

Si se inserta un nuevo actor o actriz de voz sin fecha de nacimiento, MongoDB si permite esta inserción, esto debido a que mongoDB almacena documentos sin una estructura fija, un documento

```
PS C:\Users\danie\OneDrive\Desktop\Tareas_BDA\Tarea1> docker exec -ti mongo1 bash
root@1a3bc1fad67:/# mongosh -port 30001
Current Mongosh Log ID: 6aa0a5c669109c8a3b36a87c
Connecting to:      mongodb://127.0.0.1:30001/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.10.0
Using MongoDB:      8.0.29
Using Mongosh:       2.10.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

The server generated these startup warnings when booting
2026-09-08T23:45:10.092+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-09-08T23:45:10.092+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-08T23:45:10.092+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-08T23:45:10.092+00:00: We suggest setting the contents of sysfsFile to 0.
2026-09-08T23:45:10.092+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----
my-replica-set [direct: primary] test> 
PS C:\Users\danie\OneDrive\Desktop\Tareas_BDA\Tarea1> docker stop mongo1
mongo1
```

Figura 5: Proceso encontrar y botar nodo primario.

podría no tener los mismos campos que sus pares y aun así ser almacenado. Pero esto puede llevar a problemas, por ejemplo, consultas inconsistentes al momento de hacer un `.find()`, o también si se crea un índice con respecto a este atributo, puede traer inconsistencias al momento de funcionar.

A pesar de que puede traer problemas, la realidad es que en este caso no se suele filtrar o usar demasiado el campo `FechaNacimiento`, además de que una de las cualidades que tiene MongoDB es que acepte este tipo de información. Pero, si aún así se desea tener una cantidad de campos mínimos como los que se estipulan, entonces se puede validar la estructura con MongoDB y si falta un campo obligatorio, entonces fallara la inserción.

4.2. Resiliencia de la base de datos

Como se ve en la imagen 5, `mongo1` era el nodo primario y lo botamos para ver la reacción. Al hacer la consulta en mongoDB, efectivamente la consulta era procesada y mostraba resultados. Esto se debe a la forma de replicación que tiene, cuando un nodo se cae, uno de los secundarios adopta el lugar de primario y trabaja con los datos que tenían hasta ese momento.

4.3. Actualización masiva de documentos

Para actualizar los documentos se usaron las siguientes consultas:

```
db["Voice-actors"].updateMany(
  {"Edad": { $lt: 40 }},
  {$push: {
    "Personajes": {
      "NombrePersonaje": "Main",
      "Produccion": "Sansa_Ball_Race",
      "Rol": "principal",
      "Apariciones": 3
    }
  }},
  $mul: {"Patrimonio": 1.32}
)
db["Voice-actors"].updateMany(
  {"Edad": { $gte: 40 }},
  {$push: {
    "Personajes": {
      "NombrePersonaje": "second",
      "Produccion": "Sansa_Ball_Race",
      "Rol": "secundario",
      "Apariciones": 1
    }
  }},
  $mul: {"Patrimonio": 1.32}
)
```

```

db["Voice-actors"].find({
  "Edad": {$lt: 40},
  "Personajes": {
    $elemMatch: {"Rol": "principal", "Generos": "Accion"}
  }},
  {"_id": 0, "Nombre": 1, "Edad": 1, "Patrimonio": 1, "Personajes": 1
}).sort({"Edad": 1}).limit(10)
{
  Nombre: 'Yuki Nakamura',
  Patrimonio: 60000,
  Personajes: [
    {
      NombrePersonaje: 'Anyu Forger',
      Produccion: 'Spy x Family',
      TipoProduccion: 'profesional',
      Rol: 'principal',
      Apariciones: 37,
      Generos: [
        'Comedia',
        'Accion',
        'Drama'
      ]
    }
  ],
  Edad: 22
}

```

Figura 6: Documentos pre update (menores de 40 años).

Se puede ver en [6](#) y [7](#) que se agrego el personaje y no necesariamente con todos los atributos que principalmente se habia pedido y aún asi logro insertarlo. Lo mismo pasa en [8](#) y [9](#), Además que se puede ver que funciona correctamente el aumento de 32% en sus patrimonios.

5. Parte 4: Cierre

Aprovecho esta sección para mencionar que las distintas carpetas dentro de **Tarea1** son información que se utilizo para hacer las tablas, así como también material adicional de consultas largas. Se guardo los JSON de los explain para buscar mejor los datos.

```

> db["Voice-actors"].find({
  "Edad": {$lt: 40},
  "Personajes": {
    $elemMatch: {"Rol": "principal", "Generos": "Accion"}
  },
  {"_id": 0, "Nombre": 1, "Edad": 1, "Patrimonio": 1, "Personajes": 1
}).sort({"Edad": 1}).limit(10)
< {
  Nombre: 'Yuki Nakamura',
  Patrimonio: 79200,
  Personajes: [
    {
      NombrePersonaje: 'Anya Forger',
      Produccion: 'Spy x Family',
      TipoProduccion: 'profesional',
      Rol: 'principal',
      Apariciones: 37,
      Generos: [
        'Comedia',
        'Accion',
        'Drama'
      ]
    },
    {
      NombrePersonaje: 'Main',
      Produccion: 'Sansa Ball Race',
      Rol: 'principal',
      Apariciones: 3
    }
  ],
  Edad: 22
}

```

Figura 7: Documentos post update (menores de 40 años).

```

> db["Voice-actors"].find({
  "Edad": {$lt: 30},
  "Personajes": {
    $elemMatch: {"Rol": "secundario", "Generos": "Accion"}
  }},
{"_id": 0, "Nombre": 1, "Edad": 1, "Nacionalidad": 1, "Personajes": 1
}).sort({"Edad": 1}).limit(10)
< {
  Nombre: 'Lucas Bittencourt',
  Nacionalidad: 'Brasil',
  Personajes: [
    {
      NombrePersonaje: 'Miguel O'Hara / Spider-Man 2099',
      Produccion: 'Spider-Man: A Través del Spider-Verso',
      TipoProduccion: 'profesional',
      Rol: 'secundario',
      Apariciones: 1,
      Generos: [
        'Accion',
        'Superheroes',
        'Aventura'
      ]
    }
  ],
  Edad: 28
}
{
  Nombre: 'Bartek Nowak',
  Nacionalidad: 'Polonia',
  Personajes: [
    {
      NombrePersonaje: 'Geralt (voz adicional)',
      Produccion: 'The Witcher: Ronin',
      TipoProduccion: 'profesional',
      Rol: 'secundario',
      Apariciones: 1,
      Generos: [
        'Fantasia',

```

Figura 8: Documentos pre update (mayores o igual a 40 años).

```

> db["Voice-actors"].find({
  "Edad": {$gte: 40},
  "Personajes": {
    $elemMatch: {"Rol": "secundario", "Generos": "Accion"}
  }},
  {"_id": 0, "Nombre": 1, "Edad": 1, "Patrimonio": 1, "Personajes": 1
}).sort({"Edad": 1}).limit(10)
< {
  Nombre: 'Cristina Vee',
  Patrimonio: 3833280,
  Personajes: [
    {
      NombrePersonaje: 'Sailor Moon',
      Produccion: 'Sailor Moon Crystal',
      TipoProduccion: 'profesional',
      Rol: 'principal',
      Apariciones: 39,
      Generos: [
        'Fantasia',
        'Romance',
        'Accion'
      ]
    },
    {
      NombrePersonaje: 'Killer Queen',
      Produccion: 'JoJo's Bizarre Adventure',
      TipoProduccion: 'profesional',
      Rol: 'secundario',
      Apariciones: 15,
      Generos: [
        'Accion',
        'Sobrenatural'
      ]
    },
    {
      NombrePersonaje: 'second',
      Produccion: 'Sansa Ball Race',
      Rol: 'secundario',

```

Figura 9: Documentos post update (mayores o igual a 40 años).